AI Smart Bug Analyzer & Fix Advisor

Milestone 1 Documentation (30 June – 9 July)

Project Title:AI Smart Bug Analyzer & Fix Advisor

Team:Group 2

1. Objective

The objective of this milestone is to study the defect analysis process, understand Retrieval-Augmented Generation (RAG), analyze historical bug datasets, and build the initial Bug Submission Module that will later be integrated with an AI-based bug analysis and fix recommendation system.

2. Understanding the Problem Statement

Modern software projects contain thousands of historical bug reports. Developers often spend significant time searching for similar issues and identifying possible fixes.

The AI Smart Bug Analyzer aims to:

- Analyze incoming bug reports.
- Search similar historical bugs.
- Predict bug severity.
- Suggest possible fixes.
- Reduce debugging time using AI.

3. Study of Defect Analysis Workflow

The software defect lifecycle was studied to understand how bugs move through different stages.

Bug Identified
↓
Bug Submitted
↓
Bug Assigned
↓
Bug Analysis

↓
Fix Applied
↓
Testing
↓
Verification
↓
Closed
This workflow helped in understanding where the AI Smart Bug Analyzer fits into the software development process.

4. Study of RAG (Retrieval-Augmented Generation)

The RAG architecture was studied because it allows the system to retrieve relevant historical bug reports before generating fix suggestions.

Workflow:

User Bug Report
↓
Generate Embedding
↓
Vector Database Search
↓
Retrieve Similar Bugs
↓
LLM Generates Fix Suggestion

Benefits:

- Uses previous bug knowledge.
- Produces more accurate suggestions.
- Reduces hallucination.

5. Study of Semantic Similarity

Semantic similarity enables the system to compare the meanings of bug reports instead of relying only on exact keyword matching.

Example:

Bug A:

Application crashes during login.

Bug B:
Login page closes unexpectedly.

Meaning → Same
Words → Different

This concept is essential for retrieving relevant historical defects.

6. Dataset Collection

The Bugzilla historical bug dataset was collected and analyzed.

Project directory:

bugzilla bug reports/

corpus (fixsev).txt

embedding.npy

fix.csv

fix_train.csv

fix_test.csv

sev.csv

sev_train.csv

sev_test.csv

vocab.lst

7. Dataset Exploration

corpus (fixsev).txt

Contains raw historical bug reports.

Information observed:

- Bug descriptions

- Operating system
- Developer comments
- Additional details
- Resolution status
- Verification information

Purpose:

Used as the primary knowledge source for retrieval.

---

embedding.npy

Stores vector embeddings of historical bug reports.

Purpose:

Allows fast semantic similarity search.

---

fix.csv

Dataset used for bug fix prediction.

Contains bug descriptions and corresponding fix labels.

---

fix_train.csv

Training data used to train the bug fix prediction model.

---

fix_test.csv

Testing dataset used for model evaluation.

---

sev.csv

Dataset used for severity prediction.

---

sev_train.csv

Training data for severity classification.

---

sev_test.csv

Testing data for severity classification.

---

vocab.lst

Contains vocabulary generated from the corpus.

Used during preprocessing and vectorization.

---

8. Understanding Bug Report Structure

Each bug report generally contains:

- Title
- Description
- Steps to reproduce
- Environment
- Error logs
- Stack trace
- Expected result
- Actual result
- Resolution

Understanding this structure is necessary for AI-based analysis.

---

9. Bug Submission Module Development

A basic bug submission interface was developed using Python.

Features:

- Enter bug title
- Enter bug description
- Paste stack trace
- Upload bug report files

Purpose:

Collect structured bug reports for further AI processing.

10. System Architecture Designed

User
↓
Bug Submission Module
↓
Text Processing
↓
Embedding Generation
↓
Vector Database
↓
Similarity Search
↓
AI Fix Advisor
↓
Suggested Solution

---

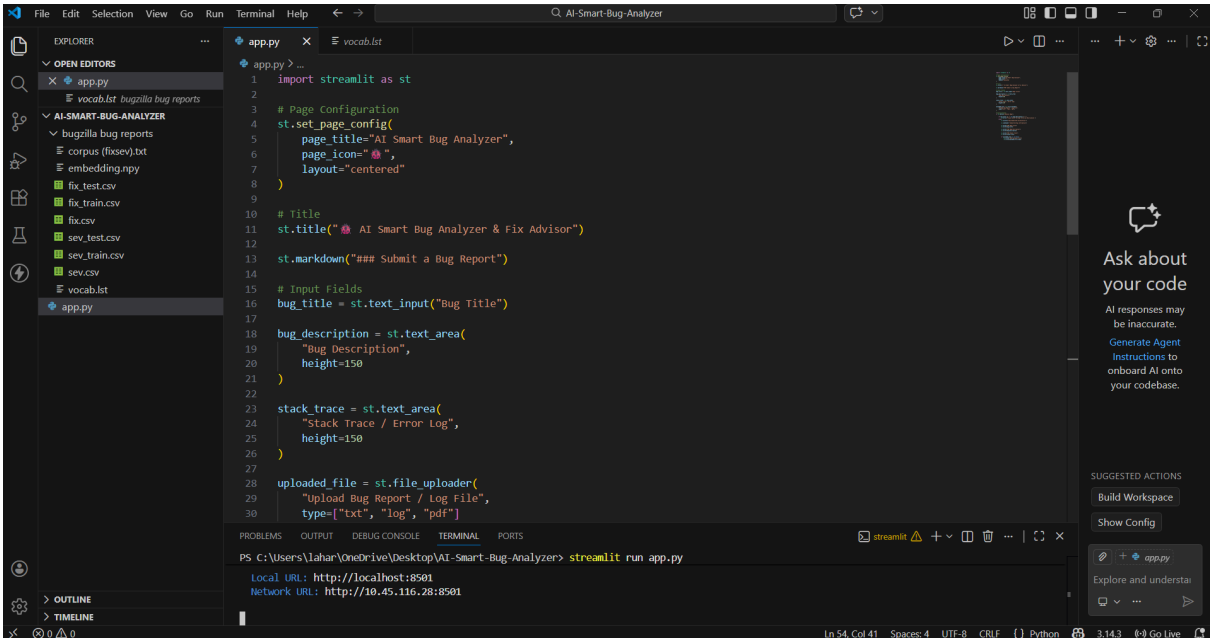
11. Technologies Studied

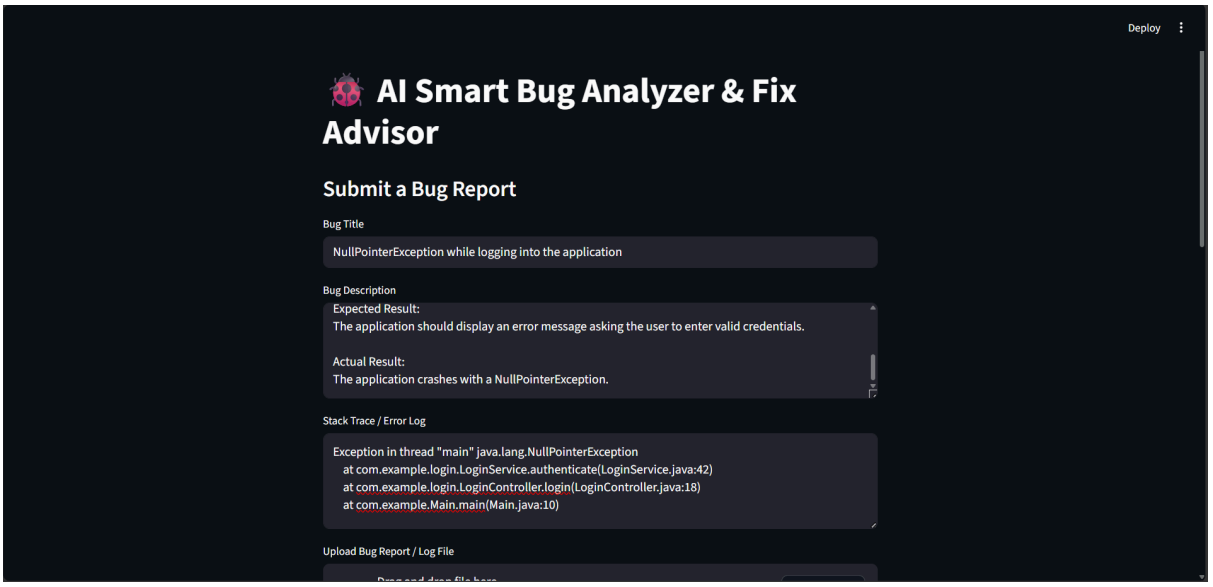
Technology	Purpose
Python	Backend development
Streamlit	User Interface
NumPy	Numerical processing
Bugzilla Dataset	Historical bugs
Embeddings	Semantic representation
RAG	Retrieval-based AI

Vector Database	Similarity search

12. Outcomes Achieved

- Studied software defect workflow.
- Understood Retrieval-Augmented Generation.
- Learned semantic similarity concepts.
- Explored Bugzilla historical datasets.
- Understood the purpose of each dataset file.
- Designed the system architecture.
- Developed the Bug Submission Module.
- Prepared the dataset for future AI integration.





GITHUB REPO LINK:https://github.com/Lahari-333/AI-Smart-Bug-Analyzer-Fix-Advisor.git

Dataset Source:

Bugzilla Historical Bug Reports
(Source: Kaggle)